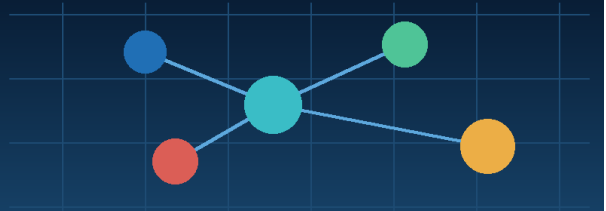


CORE PLATFORM FRAMEWORK

OFFICIAL PRODUCT DOCUMENTATION

Build · Run · Recover · Audit

master 64fd08d96392 | source 4c4248a12e69 | 186 requirements



CORE PLATFORM FRAMEWORK

데이터베이스표준서

Schema · SQL · Migration · Runtime Lock · Backup/Restore · Reconciliation

항목	기준
대상	DBA, 개발자, 플랫폼 운영자
정보 역할	DB 설계·배포·복구 정보
기준 Repository	freeangelsun/202412_01_CPF · master
기준 Commit	64fd08d963927860e8d023403dfa276931801ee5
요구사항 정보	CPF_FINAL_TARGET_REQUIREMENTS.md · 186 개
현행화 Source	4c4248a12e699c07f9f5fb11fbb33b97ca04077d (`07_16`, canonical currentization source)

이 문서는 현재 구현의 상태 보고가 아니라, CPF 제품 사용자가 업무를 끝내기 위해 따라야 할 완성 기준 사용 계약을 중심으로 작성한다. 실제 CPF Symbol/Path 는 CURRENT SOURCE, 구현 독립 예시는 REFERENCE 로 구분한다.

문서 읽는 법과 사실 구분

표기	의미	사용 규칙
CURRENT SOURCE	최신 master 에서 직접 확인한 실제 Class/API/Route/Property/SQL/Test/경로	복사·검색 가능한 실제 이름으로 사용한다.
PRODUCT CONTRACT	CPF 가 완료된 제품으로서 제공해야 하는 사용·운영 계약	현재 구현량 때문에 축소하지 않는다.
REFERENCE	사용자가 그대로 응용할 수 있도록 제공하는 완성 기준 코드·설정·절차 예시	실제 CPF 내부 Symbol 인 것처럼 표현하지 않는다.

본문은 기능을 찾는 데서 끝나지 않는다. 적용 가능한 범위에서 선택 기준 → 준비 → 구현/설정 → 실행 → 정상 결과 → 실패/부분 실패 → 복구·대사 → 보안·감사 → Test → 운영 인계 순서로 설명한다.

1. 지원 Vendor/Schema

Vendor	지원 기준
MariaDB	native install/migration/rollback/runtime query
PostgreSQL	native dialect/lock/index/sequence
Oracle	native tablespace/type/sequence/lock

Schema	Owner
cpfDB	core platform
cmnDB	common
admDB	ADM
bzaDB	BZA
batDB	Batch
refDB	Reference
Generated	Generator manifest domain owner

2. Naming/Data Type 표준

대상	표준
Table	owner/업무 의미가 드러나는 snake_case
PK	업무 ID 또는 명시적 surrogate key
Version	optimistic lock version_no
Timestamp	created_at/updated_at + timezone 정책
State	enumerated contract + transition owner
Hash	SHA-256 hex/bytes 명확화
PII	classification/masking/retention/crypto policy
Index	query predicate/sort/unique/idempotency 를 근거로 생성

3. Generator-first Lifecycle

● ● ● FLOW · 3. Generator-first Lifecycle

Requirement/Data Model

→ Canonical Schema/Metadata/Runtime Query Contract

→ Generator + Golden Template

→ MariaDB / PostgreSQL / Oracle native source

→ Install / Seed / Migration / Rollback / Runtime Pack

→ Java Consumer / Test

→ Fresh Runtime Evidence

4. Fresh Install

1. 대상 DB/Schema 에서 CPF object 0 건 확인
2. least-privilege owner/runtime account 분리
3. vendor install pack checksum 검증
4. DDL 적용
5. mandatory seed idempotent 적용
6. schema manifest/index/FK/version verify
7. runtime service user create-read-update-delete/query 권한 probe
8. reinstall idempotency 확인

5. Migration 표준

● ● ● CODE · V20260808_001__pay_transfer_add_request_hash.sql

```
-- V20260808_001__pay_transfer_add_request_hash.sql
-- Requirement: PAY-IDEMPOTENCY
-- Owner: Payment Domain
-- Lock impact: metadata lock < approved window
-- Backfill: bounded batches, resumable
-- Compatibility: old app reads nullable until promotion
-- Rollback/Forward recovery: remove index after old binary restored; retain data column
```

금지	이유
기존 Flyway migration 수정	이미 적용된 history/hash 훼손
Vendor SQL 복사 후 문자열 치환	native semantics/index/lock 차이를 숨김
drift 를 skip	잘못된 schema 를 정상으로 위장
데이터 삭제 rollback 자동화	backup/승인/보존 정책 없이 파괴적

6. Runtime Query/SQL

운영 SQL 은 Java literal 로 풀어 놓지 않고 Owner Query ID 와 Vendor resource 를 관리한다. value 는 binding 하고 identifier 는 canonical metadata/allowlist 에서만 선택한다.

● ● ● SQL · 운영 SQL 은 Java literal 로 풀어 놓지 않고 Owner Query ID 와 Vendor resource 를

```
-- Stable paging example
SELECT transfer_id, state, updated_at
FROM pay_transfer
WHERE updated_at >= :fromTs
  AND (:state IS NULL OR state = :state)
ORDER BY updated_at DESC, transfer_id DESC
FETCH FIRST :limit ROWS ONLY;
```

7. Lock/Concurrency

방법	사용
Optimistic version	관리/업무 record 동시 수정
DB row lock	짧은 single DB critical section
CpfLockManager lease/fence	multi-instance long/remote coordination
Unique key	idempotency/business uniqueness
Advisory lock	Vendor-specific 라면 SPI/adapter 와 fallback 정책 명시

8. Outbox/Inbox/Saga/TCC DB Model

Outbox

대표 필드: event_id, tx_id, state, lease_owner, fence, attempt, payload_hash, available_at, published_at

Index 는 pending/recovery scan 과 unique/idempotency query 를 근거로 설계하고 retention/archive/purge 를 포함한다.

Inbox

대표 필드: message_id, consumer, request_hash, state, version, processed_at

Index 는 pending/recovery scan 과 unique/idempotency query 를 근거로 설계하고 retention/archive/purge 를 포함한다.

Saga

대표 필드: saga_id, step, state, compensation_state, version, tx_id

Index 는 pending/recovery scan 과 unique/idempotency query 를 근거로 설계하고 retention/archive/purge 를 포함한다.

TCC

대표 필드: reservation_id, request_id, state, expires_at, version, tx_id

Index 는 pending/recovery scan 과 unique/idempotency query 를 근거로 설계하고 retention/archive/purge 를 포함한다.

9. Backup/Restore

9. backup point/RPO/RTO/retention/encryption 정의
10. backup integrity/hash 와 restore 권한 확인
11. application writes/quiesce 요구 결정
12. DB restore
13. schema/Flyway version 검사
14. idempotency/outbox/inbox/batch/audit 상태 검사
15. 업무 count/amount/version/hash 대사
16. application compatibility/readiness/business probe
17. traffic reopen + post-restore monitoring

10. Drift/Reconciliation

Drift	Action
Object missing/extra	install/migration history 와 비교, 임의 create/drop 금지
Index/FK mismatch	canonical metadata 기준 영향 분석 후 migration
Schema version ahead/behind	호환 binary 선택 또는 controlled migration
Outbox/Inbox mismatch	provider/consumer result 대사
Batch metadata mismatch	Spring Batch IDs + CPF ledger 대사
Audit chain break	tamper/partial-write incident 로 처리

11. Vendor 별 체크

MariaDB

검증	내용
DDL	type/default/identity/sequence/native syntax
Index	sort/null/cardinality/size
Lock	isolation/row lock/deadlock/timeout
Migration	online/metadata lock/DDL transaction semantics
Backup	vendor tool + checksum/point-in-time
Restore	new isolated DB 에서 schema+data+runtime probe

PostgreSQL

검증	내용
DDL	type/default/identity/sequence/native syntax
Index	sort/null/cardinality/size
Lock	isolation/row lock/deadlock/timeout
Migration	online/metadata lock/DDL transaction semantics
Backup	vendor tool + checksum/point-in-time
Restore	new isolated DB 에서 schema+data+runtime probe

Oracle

검증	내용
DDL	type/default/identity/sequence/native syntax
Index	sort/null/cardinality/size
Lock	isolation/row lock/deadlock/timeout
Migration	online/metadata lock/DDL transaction semantics
Backup	vendor tool + checksum/point-in-time
Restore	new isolated DB 에서 schema+data+runtime probe

Source Trace

구분	Repository 경로	확인 포인트
Canonical requirement	CPF_FINAL_TARGET_REQUIREMENTS.md	3 vendor/generator-first/fresh lifecycle
Vendor owner	cpf-tools/db/vendor/<vendor>	install/migration/rollback/runtime pack

구분	Repository 경로	확인 포인트
ADM Config	cpf-admin/src/main/resources/ application.yml	admDB connection
Gateway Config	cpf-gateway/src/main/resources/ application.yml	cpfDB connection

12. DB Artifact 상세 설계 예

12.1 Idempotency Ledger

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.1 Idempotency Ledger 실행/대사 예

```
-- [REFERENCE] key/scope/hash conflict 를 보존
INSERT INTO cpf_idempotency(scope_key,idempotency_key,request_hash,status,created_at)
VALUES (:scope,:key,:hash,'STARTED',CURRENT_TIMESTAMP);
SELECT request_hash,status,response_hash FROM cpf_idempotency
WHERE scope_key=:scope AND idempotency_key=:key;
```

설계	내용
대표 Column	scope,key,request_hash,state,result_ref,expires_at,version
Index/Constraint	UNIQUE(scope,key), state+expires index
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.2 Outbox

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.2 Outbox 실행/대사 예

```
-- [REFERENCE] publish 후보 claim
SELECT event_id,transaction_id,payload,row_version FROM cpf_outbox
WHERE status='READY' AND next_attempt_at<=CURRENT_TIMESTAMP
ORDER BY created_at FETCH FIRST 100 ROWS ONLY;
UPDATE cpf_outbox SET status='CLAIMED', owner_id=:owner, row_version=row_version+1
WHERE event_id=:eventId AND status='READY' AND row_version=:expectedVersion;
```

설계	내용
대표 Column	event_id,tx_id,state,lease_owner,fence,attempt,available_at,payload_hash
Index/Constraint	state+available_at, tx_id
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.3 Inbox

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.3 Inbox 실행/대사 예

```
-- [REFERENCE] consumer dedup 은 message/business key unique 로 닫는다
INSERT INTO cpf_inbox(message_id,consumer_id,payload_hash,status,received_at)
VALUES (:messageId,:consumerId,:hash,'RECEIVED',CURRENT_TIMESTAMP);
-- duplicate key => 동일 hash 면 no-op, 다른 hash 면 conflict/audit
```

설계	내용
대표 Column	message_id,consumer,state,request_hash,processed_at,version
Index/Constraint	UNIQUE(message_id,consumer), state
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.4 Saga

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.4 Saga 실행/대사 예

```
-- [REFERENCE] Saga step/compensation 상태를 영속화
UPDATE cpf_saga_step SET status=:status, attempt_no=attempt_no+1,
operation_id=:operationId, updated_at=CURRENT_TIMESTAMP
WHERE saga_id=:sagaId AND step_no=:stepNo AND row_version=:expectedVersion;
```

설계	내용
대표 Column	saga_id,step,state,comp_state,tx_id,version
Index/Constraint	state+updated_at, tx_id
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.5 TCC

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.5 TCC 실행/대사 예

```
-- [REFERENCE] Try reservation 과 Confirm/Cancel 을 CAS 로 보호
UPDATE cpf_tcc_reservation SET state='CONFIRMED', row_version=row_version+1
WHERE reservation_id=:id AND state='TRIED' AND row_version=:expectedVersion;
-- 0 row => stale/duplicate/state conflict 를 구분해 result query
```

설계	내용
대표 Column	reservation_id,request_id,state,expires_at,version,tx_id
Index/Constraint	UNIQUE(request_id), state+expires_at
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.6 Lock Lease

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.6 Lock Lease 실행/대사 예

```
-- [REFERENCE] stale writer 를 fencing token 으로 차단
UPDATE cpf_lock_lease SET owner_id=:owner, lease_until=:until,
fencing_token=fencing_token+1
WHERE lock_key=:key AND (lease_until<CURRENT_TIMESTAMP OR owner_id=:owner);
SELECT fencing_token,lease_until FROM cpf_lock_lease WHERE lock_key=:key;
```

설계	내용
대표 Column	resource_key,owner_id,request_id,fence,owner_epoch,version,lease_until,state
Index/Constraint	PK resource_key, state+lease_until
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.7 Transaction Segment

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.7 Transaction Segment 실행/대사 예

```
-- [REFERENCE] transaction lineage 조회
SELECT transaction_id,segment_id,parent_segment_id,source_module,target_module,
status,started_at,ended_at FROM cpf_transaction_segment
WHERE transaction_id=:transactionId ORDER BY started_at,segment_id;
```

설계	내용
대표 Column	transaction_id,segment_id,parent,attempt,instance,result,error,elapsed
Index/Constraint	transaction_id+time, trace_id
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.8 Batch Operation

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.8 Batch Operation 실행/대사 예

```
-- [REFERENCE] JobExecution/StepExecution 과 CPF operation 을 함께 대사
SELECT operation_id,job_name,job_instance_id,job_execution_id,status,row_version
FROM cpf_batch_operation WHERE job_execution_id=:executionId;
-- restart 전에 Spring Batch metadata 의 identifying parameter/checkpoint 와 함께 확인
```

설계	내용
대표 Column	job/execution/step/partition/item/attempt/state/fence/tx_id
Index/Constraint	execution/state, tx_id
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

12.9 Audit Chain

아래 SQL 은 제품 계약을 설명하기 위한 REFERENCE 입니다. 현재 Vendor SQL 을 적용할 때는 해당 migration/runtime query 파일의 실제 문법을 우선합니다.

● ● ● SQL · 12.9 Audit Chain 실행/대사 예

```
-- [REFERENCE] tamper-evident chain 검증 입력
SELECT audit_id,previous_hash,record_hash,actor_id,action_type,target_id,reason
FROM cpf_audit_chain WHERE chain_id=:chainId ORDER BY sequence_no;
-- canonical payload 재해시 후 previous_hash 연결과 head hash 일치 판정
```

설계	내용
대표 Column	record_id,sequence,canonical_hash,previous_hash,current_hash,signature,key_version
Index/Constraint	sequence unique, actor/time/target
Concurrency	optimistic version/unique/lease-fence 중 contract 에 맞게
Retention	terminal state/법규/감사 요구 기준 archive/purge
PII/Secret	원문 최소화/masking/encryption/reference
Migration	3 Vendor native artifact + immutable migration
Recovery	partial/unknown row 를 scan 할 runtime query
ADM	필요 projection 은 Owner Query 로 노출

13. Restore 후 대사 Query 설계

13.1 업무 건수

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.1 업무 건수 대사 Query

```
-- [REFERENCE] 기준시점 업무 건수 대사
SELECT business_date,status,COUNT(*) AS cnt
FROM business_order WHERE business_date=:businessDate
GROUP BY business_date,status ORDER BY status;
-- backup source / restored target 를 같은 기준시점으로 비교
```

Restore 직후 businessDate/state 별 count 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

13.2 금액

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.2 금액 대사 Query

```
-- [REFERENCE] 건수만 같아도 금액이 다르면 복구 종료 금지
SELECT business_date,currency,COUNT(*) cnt,SUM(amount) total_amount
FROM business_order WHERE business_date=:businessDate
GROUP BY business_date,currency ORDER BY currency;
```

Restore 직후 업무 day/currency 별 sum 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

13.3 Idempotency

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.3 Idempotency 대사 Query

```
-- [REFERENCE] 처리 완료와 중간 상태를 분리
SELECT status,COUNT(*) cnt FROM cpf_idempotency
WHERE created_at BETWEEN :from AND :to GROUP BY status;
SELECT scope_key,idempotency_key,request_hash,status FROM cpf_idempotency
WHERE status IN ('STARTED','UNKNOWN_RESULT');
```

Restore 직후 processing/unknown/expired count 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

13.4 Outbox

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.4 Outbox 대사 Query

```
-- [REFERENCE] 복구 후 publish backlog/claim 고착을 확인
SELECT status,COUNT(*) cnt,MIN(created_at) oldest FROM cpf_outbox GROUP BY status;
SELECT event_id,owner_id,next_attempt_at,last_error FROM cpf_outbox
WHERE status IN ('READY','CLAIMED','FAILED') ORDER BY created_at;
```

Restore 직후 PENDING/UNKNOWN/DEAD count 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

13.5 Inbox

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.5 Inbox 대사 Query

```
-- [REFERENCE] dedup conflict 와 미완료 consumer 처리 확인
SELECT status,COUNT(*) cnt FROM cpf_inbox GROUP BY status;
SELECT message_id,consumer_id,payload_hash,status,last_error FROM cpf_inbox
WHERE status NOT IN ('COMPLETED','DUPLICATE') ORDER BY received_at;
```

Restore 직후 PROCESSING/UNKNOWN count 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

13.6 Batch

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.6 Batch 대사 Query

```
-- [REFERENCE] CPF operation 과 Spring Batch metadata 를 함께 대사
SELECT job_name,status,COUNT(*) cnt FROM cpf_batch_operation
WHERE business_date=:businessDate GROUP BY job_name,status;
-- JobRepository BATCH_JOB_EXECUTION / BATCH_STEP_EXECUTION 상태와 교차 확인
```

Restore 직후 running/unknown/ghost execution 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

13.7 Audit

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.7 Audit 대사 Query

```
-- [REFERENCE] chain 단절 후보 확인
SELECT chain_id,sequence_no,previous_hash,record_hash FROM cpf_audit_chain
WHERE chain_id=:chainId ORDER BY sequence_no;
-- application 에서 canonical hash 를 재계산해 head hash 까지 연속성 검증
```

Restore 직후 hash chain verify count/break position 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

13.8 Schema

Restore/DR 종료 판정은 단일 Health 가 아니라 아래 대사 결과와 업무 Owner 기준을 함께 사용합니다.

● ● ● SQL · 13.8 Schema 대사 Query

```
-- [REFERENCE] migration history 와 대상 object 를 함께 확인
SELECT version,description,success,installed_on FROM flyway_schema_history ORDER BY installed_rank;
-- Vendor catalog 에서 expected table/index/constraint 존재 여부를 verify script 와 비교
-- drift 가 있으면 원인/영향/rollback 가능성을 기록한 뒤 변경 승인
```

Restore 직후 manifest object/index/FK/version hash 를 백업 시점/업무 원장/Provider 상태와 비교한다. 불일치는 traffic open 전에 분류하고 필요한 reconcile 을 수행한다.

14. Core Transaction-Security DB Pack

현재 DB3 Vendor Pack 에는 broker inbox dedup hardening 과 core transaction/security hardening migration 이 추가되어 있다. Vendor 별 install/source/migration/rollback/runtime/verify 를 같은 변경 단위로 대조한다.

Vendor/Pack	Canonical Path	역할
MariaDB V110	cpf-tools/db/vendor/mariadb/migration/V110__broker_inbox_dedup_hardening.sql	Inbox/Dedup hardening
MariaDB V111	cpf-tools/db/vendor/mariadb/migration/V111__core_tx_security_hardening.sql	Core transaction/security hardening
PostgreSQL V110	cpf-tools/db/vendor/postgresql/migration/V110__broker_inbox_dedup_hardening.sql	Inbox/Dedup hardening
PostgreSQL V111	cpf-tools/db/vendor/postgresql/migration/V111__core_tx_security_hardening.sql	Core transaction/security hardening
Oracle V110	cpf-tools/db/vendor/oracle/migration/V110__broker_inbox_dedup_hardening.sql	Inbox/Dedup hardening
Oracle V111	cpf-tools/db/vendor/oracle/migration/V111__core_tx_security_hardening.sql	Core transaction/security hardening
Runtime Query	*/runtime/messaging/broker_inbox_queries.sql	Inbox runtime query
Runtime Query	*/runtime/transaction/core_tx_security_queries.sql	Transaction/security runtime query
Verify	*/verify/110_* + 111_*	적용 후 필수 Object/Contract 확인
Rollback	*/rollback/R110_* + R111_*	rollback 또는 forward recovery 판단 근거

16. 실행형 DB 표준 워크북

16.1 Migration/Verify/Rollback 한 세트

● ● ● SQL · V111__core_tx_security_hardening.sql

```
-- PRODUCT SOURCE 에는 vendor 별 V111 파일이 존재한다.  
-- 변경은 owner/table/index/constraint 와 Runtime query 를 함께 검증한다.  
ALTER TABLE <owner_table> ADD COLUMN <required_column> ...;  
CREATE INDEX <bounded_index> ON <owner_table>(...);
```

● ● ● SQL · VERIFY

```
SELECT table_name, column_name, data_type  
FROM information_schema.columns  
WHERE table_name = '<owner_table>';
```

● ● ● CODE · ROLLBACK

```
-- destructive rollback 가능성/데이터 손실을 먼저 판정한다.  
-- 불가능하면 forward recovery 를 선택하고 이유를 기록한다.
```

16.2 UNKNOWN/Reconcile 조회 원칙

● ● ● SQL · 대사 조회

```
SELECT transaction_id, business_key, state, provider_request_id, version_no, updated_at  
FROM pay_transfer  
WHERE state = 'UNKNOWN'  
ORDER BY updated_at  
FETCH FIRST 100 ROWS ONLY;
```

DBA 금지

상태 Column 을 임의 UPDATE 하여 UNKNOWN 을 COMPLETED 로 바꾸지 않습니다. Owner result query 와 business side effect 를 대조한 뒤 승인된 Reconcile Command 를 사용합니다.

문서 기준 및 검증 범위

항목	기준
Repository / Branch	freeangelsun/202412_01_CPF / master
Work start / latest reviewed HEAD	64fd08d963927860e8d023403dfa276931801ee5 (07_17)
Latest product source change basis	4c4248a12e699c07f9f5fb11fbb33b97ca04077d (07_16 canonical currentization source)
Canonical Requirement	186 개
CURRENT SOURCE	실제 Repository 에서 확인한 Symbol/경로
PRODUCT CONTRACT	현재 구현량과 분리한 제품 목표 계약
REFERENCE	복사·응용 가능한 완성 기준 예시; CURRENT SOURCE 아님
Runtime qualification	문서 세션에서 직접 실행하지 않은 DB/Browser/Multi-instance/Fault 는 미검증